



UNIVERSIDAD
NACIONAL
DE COLOMBIA

PROBLEMA DE GESTIÓN DINÁMICA DE MEMORIA CACHE MODELADO COMO UN MDP Y APROXIMADO MEDIANTE Q-LEARNING

ESTUDIANTE:

CARLOS ANDRES RIOS ROJAS

criosro@unal.edu.co

1006797081

PROFESOR:

JORGE EDUARDO ORTIZ TRIVIÑO

UNIVERSIDAD NACIONAL DE COLOMBIA - SEDE BOGOTÁ

MODELOS ESTOCÁSTICOS Y SIMULACIÓN EN COMPUTACIÓN Y COMUNICACIONES

Febrero, 2024

Bogotá D.C – Colombia

1. Marco Teórico

1.1. Procesos Markovianos

Un proceso markoviano es un tipo de proceso estocástico en el cual la probabilidad de transición de un estado a otro depende únicamente del estado actual y no del historial previo de estados. Esta propiedad, conocida como la **propiedad de Markov**, se expresa matemáticamente como:

$$P(X_{t+1} | X_t, X_{t-1}, \dots, X_0) = P(X_{t+1} | X_t)$$

donde X_t representa el estado del sistema en el tiempo t .

Los procesos de Markov pueden clasificarse en **cadena de Markov** (cuando el espacio de estados es discreto) y **procesos de Markov en tiempo continuo**. Estos modelos han sido ampliamente utilizados en diversos campos como inteligencia artificial, telecomunicaciones, biología computacional y optimización de sistemas dinámicos.

Una de las características clave de estos procesos es la matriz de transición de probabilidades, que define la probabilidad de moverse de un estado a otro en cada paso del tiempo. En su forma más simple, una **cadena de Markov homogénea** en espacio de estados finito se modela mediante una matriz P donde cada entrada p_{ij} indica la probabilidad de transición del estado i al estado j .

Los procesos de Markov sirven como base para el estudio de **Procesos de Decisión de Markov (MDPs)**, los cuales incorporan recompensas y políticas de decisión para modelar problemas de toma de decisiones en entornos inciertos [1].

1.2. Procesos de Decisión de Markov (MDPs)

Los **Procesos de Decisión de Markov (MDPs, por sus siglas en inglés)** son un marco matemático utilizado para modelar problemas de toma de decisiones secuenciales en entornos inciertos. Un MDP se define formalmente como una tupla (S, A, P, R, γ) donde:

- S es el conjunto de **estados** del sistema, con estado inicial s_0 .

- A es el conjunto de **acciones** disponibles para el agente.
- $P(s' | s, a)$ es la **función de transición**, que indica la probabilidad de transicionar al estado s' dado que el agente toma la acción a en el estado s .
- $R(s, a, s')$ es la **función de recompensa**, que asigna un valor numérico indicando el beneficio inmediato por transicionar desde el estado s al estado s' por medio de la acción a .
- γ es el **factor de descuento**, donde $0 \leq \gamma \leq 1$, que pondera la importancia de recompensas futuras en comparación con las inmediatas.

El objetivo en un MDP es encontrar una **política óptima** $\pi^*(s)$, que es una función que asigna la mejor acción a cada estado para maximizar la recompensa acumulada esperada a lo largo del tiempo. Esta recompensa se denomina **Utilidad** y se define para todo estado s y política π de la siguiente manera:

$$U^\pi(s) = E \left[\sum_{t=0}^{\infty} \gamma^t R(S_t, \pi(S_t), S_{t+1}) \right]$$

Intuitivamente, representa qué tan bueno es un estado si se toma siempre la mejor decisión desde allí. Con lo anterior, es posible entonces extraer la política óptima así

$$\pi_s^* = \underset{\pi}{\operatorname{argmax}} U^\pi(s)$$

De manera análoga, se pueden determinar las mejores acciones para cada estado de la siguiente forma

$$\pi^*(s) = \underset{a \in A(s)}{\operatorname{argmax}} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')]$$

Y teniendo esto en cuenta, se encuentra que la utilidad de un estado es la recompensa esperada por la siguiente transición más la utilidad descontada para el próximo estado, suponiendo que el agente escoge siempre la acción óptima, es decir, sigue la política $\pi^*(s)$. De esta forma se deriva la **Ecuación de Bellman**:

$$U(s) = \max_{a \in A(s)} \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')]$$

Una cantidad muy importante es la **Función de acción-utilidad** o **Función Q** $Q(s, a)$, esta es la utilidad esperada al realizar cierta acción en un estado dado. En términos prácticos, este valor nos dice que tan bueno es realizar una acción en un estado en particular. La función Q se relaciona con las utilidades de la siguiente forma:

$$U(s) = \max_a Q(s, a)$$

$$\pi^*(s) = \operatorname{argmax}_a Q(s, a)$$

Y se define propiamente como:

$$\begin{aligned} Q(s, a) &= \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma U(s')] \\ &= \sum_{s'} P(s' | s, a) [R(s, a, s') + \gamma \max_{a'} Q(s', a')] \end{aligned}$$

Solucionar las ecuaciones de Bellman para U o Q nos brinda lo necesario para encontrar la política óptima [2].

1.3. Aprendizaje por Refuerzo (RL)

El **Aprendizaje por Refuerzo (Reinforcement Learning, RL)** es un paradigma del aprendizaje automático en el que un agente aprende a tomar decisiones óptimas mediante la interacción con un entorno, con el objetivo de maximizar una recompensa acumulada. A diferencia del aprendizaje supervisado, donde los datos de entrenamiento incluyen respuestas correctas, en RL el agente debe descubrir la mejor estrategia a través de prueba y error.

Un problema de RL se modela generalmente como un **Proceso de Decisión de Markov (MDP)**, definido por la tupla (S, A, P, R, γ) . En este contexto:

- **El agente representa la entidad que toma decisiones** ejecutando acciones $a \in A$ en diferentes estados $s \in S$.
- **El entorno proporciona la retroalimentación**, modelada mediante la función de transición de estados $P(s' | s, a)$ y la función de recompensa $R(s, a, s')$.
- **El objetivo del agente es aprender una política óptima** $\pi^*(s)$ que maximice la recompensa total esperada a lo largo del tiempo.

El aprendizaje por refuerzo está estrechamente ligado a los MDPs, ya que estos proporcionan el marco teórico para la formulación del problema de toma de decisiones secuencial. Sin

embargo, la diferencia clave entre RL y los MDPs clásicos radica en el conocimiento del modelo del entorno:

- **MDPs tradicionales:** Se asume que la función de transición $P(s'|s, a)$ y la función de recompensa $R(s, a, s')$ son conocidas. Se pueden resolver mediante **Value Iteration** o **Policy Iteration**.
- **RL:** El agente no conoce la dinámica del entorno y debe aprenderla a partir de la experiencia. Se utilizan algoritmos como **Q-learning** y **Deep Q-Networks (DQN)** para aproximar la función de valor y determinar una política óptima.

Entre los métodos más usados en RL se encuentran:

- **Métodos basados en valores:** Como **Q-learning**, que aprende una función $Q(s, a)$ para estimar el valor de una acción en un estado dado.
- **Métodos basados en políticas:** Como **Policy Gradient**, que optimiza directamente la política $\pi(a|s)$ sin necesidad de estimar funciones de valor.
- **Métodos de aprendizaje profundo:** Como **Deep Q-Networks (DQN)** y **Actor-Critic**, que combinan redes neuronales con RL para manejar espacios de estado y acción continuos o de alta dimensión.

Gracias a esta relación con los MDPs, RL ha sido aplicado con éxito en diversas áreas como robótica, juegos, control de sistemas autónomos y finanzas, logrando resultados notables en tareas complejas como el juego de Go y el control de vehículos autónomos [2].

1.4. Gestión Dinámica de Caché en un Servidor Web

Atendiendo a [3], la **gestión dinámica de caché** en un servidor web es una técnica utilizada para optimizar el rendimiento y la eficiencia en la entrega de contenido al usuario. En lugar de almacenar respuestas estáticas, este enfoque permite la actualización y reutilización inteligente de los datos en función del acceso y la relevancia de las solicitudes.

La caché es un almacenamiento temporal de recursos web (HTML, imágenes, archivos CSS/JavaScript, respuestas de API, etc.) que permite reducir la carga del servidor y mejorar los tiempos de respuesta para los clientes. Existen varios niveles de caché:

- **Caché en el navegador:** Almacena contenido en el dispositivo del usuario para reducir la cantidad de solicitudes al servidor.
- **Caché en el servidor web:** Utiliza mecanismos como **memcached** o **Redis** para almacenar respuestas de solicitudes frecuentes.
- **Caché en la red (CDN - Content Delivery Network):** Distribuye contenido en servidores geográficamente dispersos para acelerar la entrega a los usuarios finales.

La gestión dinámica de caché implica actualizar y expirar los datos almacenados según la demanda y cambios en el contenido. Algunas estrategias clave incluyen:

- **Caché basada en políticas de reemplazo:**
 - **Least Recently Used (LRU):** Elimina los elementos menos utilizados recientemente cuando se alcanza el límite de memoria.
 - **Least Frequently Used (LFU):** Descarta los elementos con menor frecuencia de uso.
 - **Time-to-Live (TTL):** Establece un tiempo de expiración para los datos en caché.
- **Caché adaptativa:** Ajusta dinámicamente el almacenamiento en caché basándose en patrones de acceso, cargas del servidor y requisitos de latencia.
- **Cache-Control y ETags:**
 - **Encabezados HTTP como Cache-Control y Expires** permiten definir reglas de almacenamiento en el cliente y proxies intermedios.
 - **ETags (Entity Tags)** ayudan a validar si un recurso ha cambiado, evitando transferencias innecesarias.
- **Caché de contenido dinámico:** Utiliza técnicas como **Edge Side Includes (ESI)** o **Varnish Cache** para gestionar partes de páginas dinámicas sin necesidad de recargar todo el contenido.

En el contexto de **Procesos de Decisión de Markov (MDPs)**, la gestión dinámica de caché puede modelarse como un problema de optimización secuencial, donde el sistema debe decidir qué elementos almacenar o descartar para minimizar el tiempo de respuesta y el uso de recursos.

1.5. Distribución de Zipf

Atendiendo a lo dicho en [4], la **distribución de Zipf** es una distribución de probabilidad discreta que modela fenómenos donde unos pocos elementos tienen una alta frecuencia de aparición, mientras que la mayoría aparecen con menor frecuencia. Se define por la función:

$$P(k) = \frac{1/k^s}{\sum_{n=1}^N 1/n^s}$$

donde:

- $P(k)$ es la probabilidad de que el elemento k ocurra
- s es el **parámetro de sesgo** que controla la desigualdad en la distribución
- N es el número total de elementos

- k es el rango del elemento (el más frecuente tiene $k = 1$, el segundo más frecuente tiene $k = 2$, etc.).

Cuando $s \approx 1$, la distribución sigue la ley de Zipf, que se observa en fenómenos como la frecuencia de palabras en un idioma, popularidad de páginas web y distribución de tráfico en redes.

La distribución de Zipf es útil para modelar cargas de trabajo en servidores web porque sigue un **patrón realista de acceso a contenidos**, donde unos pocos elementos (páginas, archivos o recursos) son solicitados con mucha frecuencia, mientras que la mayoría son accedidos raramente.

- **Modelado del tráfico web:** En sistemas de caché, la mayoría de las solicitudes se dirigen a un pequeño conjunto de archivos populares. La distribución de Zipf permite generar cargas de prueba realistas para evaluar el rendimiento de la caché y los mecanismos de balanceo de carga.
- **Evaluación de políticas de caché:** Algoritmos como **LRU (Least Recently Used)** y **LFU (Least Frequently Used)** pueden ser evaluados utilizando tráfico basado en Zipf para verificar su eficiencia en la retención de contenido relevante.
- **Optimización de servidores y CDNs:** Servicios como YouTube, Netflix y sistemas de distribución de contenido utilizan patrones Zipfianos para predecir la demanda de archivos y optimizar la replicación de contenido.

1.6. Q-Learning

El **Q-Learning** es un algoritmo de **aprendizaje por refuerzo (RL)** basado en valores que permite a un agente aprender una política óptima para la toma de decisiones en un entorno desconocido. Fue introducido por **Chris Watkins** en 1989 y es un método de **aprendizaje fuera de línea (off-policy)** que puede aprender independientemente de la estrategia seguida por el agente [5].

El algoritmo se basa en la ecuación de actualización de **Q-valores**, que representa la utilidad esperada de tomar una acción en un estado dado y seguir la política óptima a partir de ese punto. Se basa en la siguiente expresión para actualizar los valores de la función Q:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [R(s, a, s') + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- $Q(s, a)$ es el valor actual de la acción a en el estado s
- α es la tasa de aprendizaje ($0 < \alpha \leq 1$)
- $R(s, a, s')$ es la recompensa recibida al transicionar al estado s' desde el estado s por medio de la acción a

- γ es el factor de descuento ($0 \leq \gamma \leq 10$), que pondera la importancia de futuras recompensas
- s' es el nuevo estado tras ejecutar a

El Q-Learning es una técnica diseñada para resolver **Procesos de Decisión de Markov (MDPs)** cuando la dinámica del entorno es desconocida. Permite aprender la función de valor óptima $Q^*(s, a)$ que define la mejor política de decisión sin necesidad de un modelo explícito del entorno (modelo de transición) [2].

1.7. Exploración ϵ -Greedy

Atendiendo a [2], **Aprendizaje por Refuerzo (RL)**, un agente debe equilibrar dos comportamientos:

- **Exploración:** Probar nuevas acciones para descubrir cuál es la mejor a largo plazo.
- **Explotación:** Elegir la mejor acción conocida con base en la información actual.

Un agente que explora demasiado nunca aprovecha lo que ha aprendido, mientras que uno que explota demasiado puede quedar atrapado en una solución subóptima. Para manejar este equilibrio, se utilizan estrategias de exploración como **ϵ -Greedy**.

El método **ϵ -Greedy** introduce exploración aleatoria en un algoritmo de RL como **Q-Learning** o **SARSA**. Se define de la siguiente manera:

1. Con probabilidad ϵ (normalmente entre 0.01 y 0.1), el agente **elige una acción aleatoria** (exploración).
2. Con probabilidad $1 - \epsilon$, el agente **elige la mejor acción conocida** (explotación), es decir:

$$a = \arg \max_{a'} Q(s, a')$$

donde $Q(s, a)$ es la estimación de la recompensa esperada por tomar la acción a en el estado s .

En el contexto de los **Procesos de Decisión de Markov (MDPs)**, la estrategia ϵ -Greedy permite a un agente explorar el espacio de estados mientras converge a una política óptima. En **Q-Learning**, se usa para garantizar que el agente no quede atrapado en soluciones subóptimas durante el entrenamiento [6].

2. Descripción y Justificación del Problema

En los sistemas modernos de computación y comunicaciones, la gestión eficiente del caché es crucial para mejorar el rendimiento y reducir la latencia en el acceso a datos. El caché actúa como una memoria intermedia de alta velocidad, que almacena temporalmente objetos o datos frecuentemente solicitados, permitiendo respuestas más rápidas y disminuyendo la carga sobre el almacenamiento principal o las redes de comunicación. Sin embargo, debido a la capacidad limitada de estos sistemas y a la alta variabilidad en los patrones de acceso de los usuarios, es fundamental desarrollar políticas inteligentes que determinen qué datos conservar y cuáles reemplazar cuando se produzca un fallo de caché.

Este proyecto se centra en la formulación y resolución de un problema de gestión dinámica de caché utilizando Procesos de Decisión de Markov (MDP). El enfoque propuesto modela el sistema como un MDP, donde cada estado representa el contenido actual del caché junto con su latencia de recuperación si el objeto no está en caché, y las acciones corresponden a la decisión de mantener o reemplazar objetos en respuesta a las solicitudes de los usuarios. La recompensa se define en términos del costo asociado a la latencia de acceso, lo que permite que la política aprendida se enfoque en minimizar el tiempo total de respuesta. Para poder aproximar la política óptima se implementa un agente inteligente que aprenda a gestionar la memoria caché mediante Q-learning con una política ϵ -Greedy de exploración.

En este trabajo se aplica el modelo MDP tratado en el capítulo 16 de [2]. Este capítulo también trata varios métodos para hallar la política óptima para estos modelos, sin embargo, estos métodos son limitados en cuanto el espacio de estados crece. En el mismo capítulo se menciona que la política óptima de un MDP se puede aproximar mediante aprendizaje por refuerzo (tema tratado en el capítulo 23 del mismo libro). Dadas las características del problema que se trata en este trabajo, se justifica la aplicación del conocimiento en el capítulo 23 para aproximar la solución del MDP que se plantea.

3. Diseño de la Aplicación

En este trabajo tomaremos el enfoque de optimizar el rendimiento del servidor minimizando la latencia de respuesta a las peticiones de los clientes. Las características de este modelo se listan a continuación:

- El conjunto de objetos es discreto y finito.
- El modelo supone una memoria caché del servidor finita cuya capacidad se define como la cantidad máxima de recursos que puede almacenar.
- Se conocen de antemano los recursos que los clientes van solicitar, así como su latencia de recuperación.
- La probabilidad de petición de los objetos sigue una distribución Zipf.

- El servidor no realiza ninguna acción si el objeto pedido por el cliente se encuentra en la caché.
- Si ocurre un fallo en la caché (el objeto pedido por el cliente no estaba almacenado) entonces el servidor decidirá que objeto eliminar de la memoria caché para almacenar el nuevo objeto.

Inicialmente el servidor no tendrá información acerca de la frecuencia de petición para cada objeto, por lo tanto, ante las primeras peticiones este simplemente irá almacenando los objetos hasta completar la capacidad de la memoria caché. Llegado a este punto el servidor no tiene la suficiente información como para comenzar a aprender a gestionar su caché, puesto que aun no ha tenido que tomar ninguna decisión. Por lo tanto, para el modelo con el que trabajaremos el espacio de estados corresponde solamente a todas las posibles configuraciones de la memoria caché cuando ha llegado a su máxima capacidad. Una justificación para esto es el razonamiento hecho previamente, y otra es porque de esta forma se ahorra memoria al no representar los estados iniciales que no son importantes para hallar una política óptima.

Teniendo en cuenta lo anterior, la definición formal del MDP es la siguiente:

- Cada objeto almacenado en el servidor se modela como una tupla (i, l) , donde

i : = número natural que corresponde al identificador del objeto

l : = número natural que corresponde a la latencia de recuperación del objeto en milisegundos

Además, se tiene la restricción de que cada identificador es único, es decir, dados dos objetos (i_1, l_1) , (i_2, l_2) , se tiene que si $i_1 = i_2$, entonces $(i_1, l_1) = (i_2, l_2)$.

Por convención tendremos que $\theta_i = (i, l_i)$.

Definimos también Θ : = Conjunto de todos los objetos disponibles en el servidor.

- Sea n un número natural que corresponde a la capacidad del servidor, definimos el conjunto de estados

$$S = \{ \{ \theta_0, \theta_1, \dots, \theta_{n-1} \} : \theta_i \in \Theta \}$$

Cada estado indica los objetos que están almacenados en la memoria caché.

- Definamos la acción a_i : = eliminar el objeto $\theta_i \in \Theta$ de la memoria caché para almacenar el nuevo objeto. Con esto, definamos ahora el conjunto de acciones

$$A = \{a_i : i \in [0, 1, 2, \dots, |\Theta| - 1]\}$$

- Sea a_i la acción tomada y θ_i el objeto eliminado de la memoria caché, la función de recompensa R la definimos como

$$R(s, a_i, s') = -l_i$$

- Utilizaremos un factor de descuento de $\gamma = 0.9$ para que el agente considere estrategias a largo plazo. Una razón para esto es que pueden haber objetos con bajas latencias pero que sean accedidos con mucha frecuencia haciendo que la latencia acumulada afecte negativamente el rendimiento del servidor.

El modelo de transición es desconocido pero no será necesario encontrarlo para aproximar la política óptima debido a que el agente será entrenado mediante Q-learning.

Una vez definido el modelo MDP se pasa a la implementación, la cual se realiza con el lenguaje de programación Python. El programa consta principalmente de 3 clases:

- **System:** Modela el sistema al contener el conjunto de objetos Θ y también simula las peticiones de los clientes atendiendo a la distribución Zipf.
- **Agent:** Es el gestor de la memoria caché del servidor, lo cual aprende a hacer mediante aprendizaje por refuerzo (Q-learning). También genera y muestra métricas para medir su desempeño a medida que se entrena.
- **ProbGenerator:** Es una clase con métodos estáticos para la generación de probabilidades. Contiene el generador para la distribución de Zipf.

El siguiente diagrama de clases muestra la estructura de estas clases

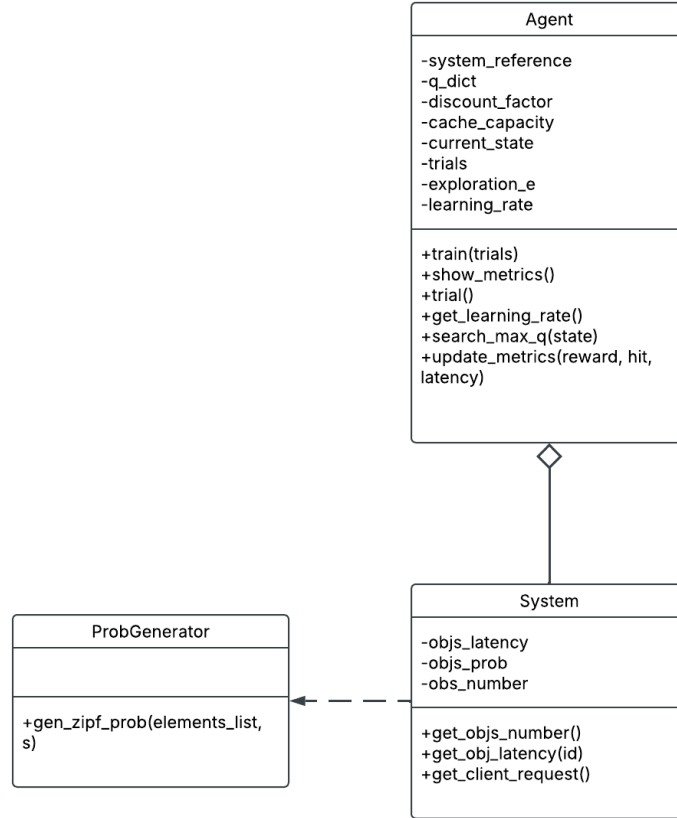


Figura 1. Diagrama de clases.

Para la implementación del agente inteligente y el aprendizaje por refuerzo mediante Q-learning se toma como base el algoritmo descrito en el capítulo 23 de [2].

function Q-LEARNING-AGENT(*percept*) **returns** an action
inputs: *percept*, a percept indicating the current state s' and reward signal r
persistent: Q , a table of action values indexed by state and action, initially zero
 N_{sa} , a table of frequencies for state–action pairs, initially zero
 s, a , the previous state and action, initially null

if s is not null **then**
 increment $N_{sa}[s, a]$
 $Q[s, a] \leftarrow Q[s, a] + \alpha(N_{sa}[s, a])(r + \gamma \max_{a'} Q[s', a'] - Q[s, a])$
 $s, a \leftarrow s', \text{argmax}_{a'} f(Q[s', a'], N_{sa}[s', a'])$
return a

Figura 2. Algoritmo Q-learning [2].

4. Manuales Técnico y de Usuario

En esta sección se recopilan los manuales técnicos y de usuario para el programa desarrollado.

4.1. Requerimientos de Software

- Python 3.11 o superior.
- Biblioteca numpy (estructuras de datos) y matplotlib (para las gráficas de las métricas)

4.2. Estructura de Archivos

Consta de un único archivo que contiene el diseño previo y la ejecución del programa.

4.3. Código Fuente

El script contiene el diseño y estructura previamente descrito. A continuación se repasa cada parte:

4.3.1. ProbGenerator

Contiene el método estático `gen_zipf_prob`, el cual recibe una lista de elementos y un parámetro de sesgo `s`. La lista debe estar ordenada de mayor a menor frecuencia esperada.

Parámetros del Método

- `elements_list`: Lista de elementos a los que se les asignará una probabilidad.
- `s`: Parámetro de sesgo que controla la distribución de las probabilidades. A mayor `s`, mayor desigualdad en la distribución.

Salida del Método

El método retorna una lista de valores normalizados, representando la distribución de probabilidad de cada elemento en función de la ley de Zipf.

Aplicación en el Proyecto

Esta funcionalidad permite asignar probabilidades de manera eficiente dentro del proceso de toma de decisiones del agente de aprendizaje por refuerzo, favoreciendo la selección de estados o acciones que siguen un comportamiento estadístico realista.

4.3.2. System

Representa el sistema y el comportamiento de las solicitudes de los clientes dentro del entorno del proyecto. Su propósito es modelar la disponibilidad de objetos, sus latencias y la probabilidad de acceso a cada uno, utilizando una distribución de Zipf para reflejar un patrón de acceso realista.

Diseño e Implementación

El sistema se inicializa a partir de una lista de objetos, donde cada objeto está definido por un identificador y su latencia. A partir de esta lista, la clase calcula:

- `objs_latency`: Lista que almacena las latencias asociadas a cada objeto.
- `objs_prob`: Distribución de probabilidad de Zipf aplicada a los objetos, lo que modela un acceso desigual donde algunos objetos son solicitados con mayor frecuencia.
- `objs_number`: Cantidad total de objetos disponibles en el sistema.

Métodos Principales

- `get_objs_number()`: Retorna el número total de objetos en el sistema.
- `get_obj_latency(id)`: Dado un identificador `id`, devuelve la latencia asociada al objeto.
- `get_client_request()`: Simula la solicitud de un cliente seleccionando un objeto de manera aleatoria, ponderada por la distribución de Zipf, reflejando así el acceso realista a los datos en sistemas de caché o almacenamiento.

Aplicación en el Proyecto

Esta clase permite modelar de manera eficiente el acceso y la latencia de los objetos en el sistema, proporcionando una base para la implementación del algoritmo de aprendizaje por refuerzo. Al simular la distribución de acceso con Zipf, se aproxima a escenarios reales donde ciertos objetos son solicitados con mayor frecuencia, optimizando la toma de decisiones en la gestión de recursos.

4.3.3. Agent

Modela la gestión de la memoria caché del sistema y aprende a optimizar su estrategia mediante el algoritmo de **Q-learning**, un método de aprendizaje por refuerzo. Su objetivo es mejorar la eficiencia del sistema mediante la optimización de la política de reemplazo de caché, maximizando las recompensas acumuladas y minimizando la latencia de las solicitudes.

Diseño e Implementación

El agente mantiene una referencia al sistema (`System`) y administra un diccionario de valores `Q` (`q_dict`), que representa las estimaciones de utilidad de cada estado y acción. Además, incorpora los siguientes elementos clave:

- `discount_factor`: Factor de descuento para evaluar recompensas futuras en la función `Q`.
- `cache_capacity`: Capacidad máxima de la caché, la cual debe ser menor o igual al número total de objetos en el sistema.

- **current_state**: Estado actual de la memoria caché, representado como un conjunto de objetos almacenados.
- **exploration_e**: Parámetro ϵ para la exploración **ϵ -greedy**, controlando el balance entre exploración y explotación.
- **learning_rate**: Tasa de aprendizaje adaptativa en función del número de iteraciones.

Funcionamiento del Algoritmo

El proceso de aprendizaje ocurre en ciclos de prueba (`trial()`), donde el agente recibe solicitudes de clientes y decide si atenderlas desde la caché o reemplazar elementos según su política de aprendizaje. Se siguen los siguientes pasos:

1. **Recepción de una solicitud**: Se selecciona un objeto con base en la distribución Zipf del sistema.
2. **Verificación en caché**: Si el objeto está en la caché, se genera un acierto (*hit*); de lo contrario, ocurre un fallo (*miss*).
3. **Toma de decisión**: Se elige una acción basada en:
 - Exploración aleatoria (ϵ -greedy).
 - Explotación del conocimiento acumulado en `q_dict`.
4. **Actualización de la función Q**: Se calcula el valor Q utilizando la ecuación de Bellman y se ajusta la política de almacenamiento.
5. **Registro de métricas**: Se acumulan las recompensas, tasas de acierto y latencias promedio para evaluar el desempeño del agente.

Visualización y Evaluación

El agente permite la visualización de métricas mediante gráficos que muestran la evolución de:

- **Recompensa promedio** (*Mean Reward Evolution*).
- **Tasa de aciertos de caché** (*Cache Hit Rate Evolution*).
- **Latencia media** (*Mean Latency Evolution*).

Aplicación en el Proyecto

Este modelo de gestión de caché basado en Q-learning permite optimizar el uso de recursos en entornos donde el acceso a datos es costoso en términos de latencia. Su implementación es clave en sistemas de almacenamiento distribuidos, redes de contenido y optimización de tráfico en infraestructuras de comunicación.

4.4. Compilación y Ejecución

4.4.1. Definición de los parámetros iniciales

- **random_seed** → Fija la semilla aleatoria para reproducibilidad.

- `discount_factor` → Define el factor de descuento de Q-learning.
- `cache_capacity` → Determina la cantidad de elementos que puede almacenar la caché.

Inicializa el sistema (System):

- Se crea una lista `objects`, donde cada objeto tiene un ID y un tiempo de latencia.
- Se instancia `System` con esta lista de objetos.

Crea un agente (Agent):

- Se inicializa el agente con el sistema `system`, el `discount_factor`, y la `cache_capacity`.

Ejecuta el entrenamiento del agente:

- Se llama a `agent.train(n)`, lo que significa que el agente entrenará por `n` iteraciones para aprender a optimizar el uso de la caché.

Para ejecutar el programa se utiliza el intérprete de Python.

4.4.2. Resultados Esperados

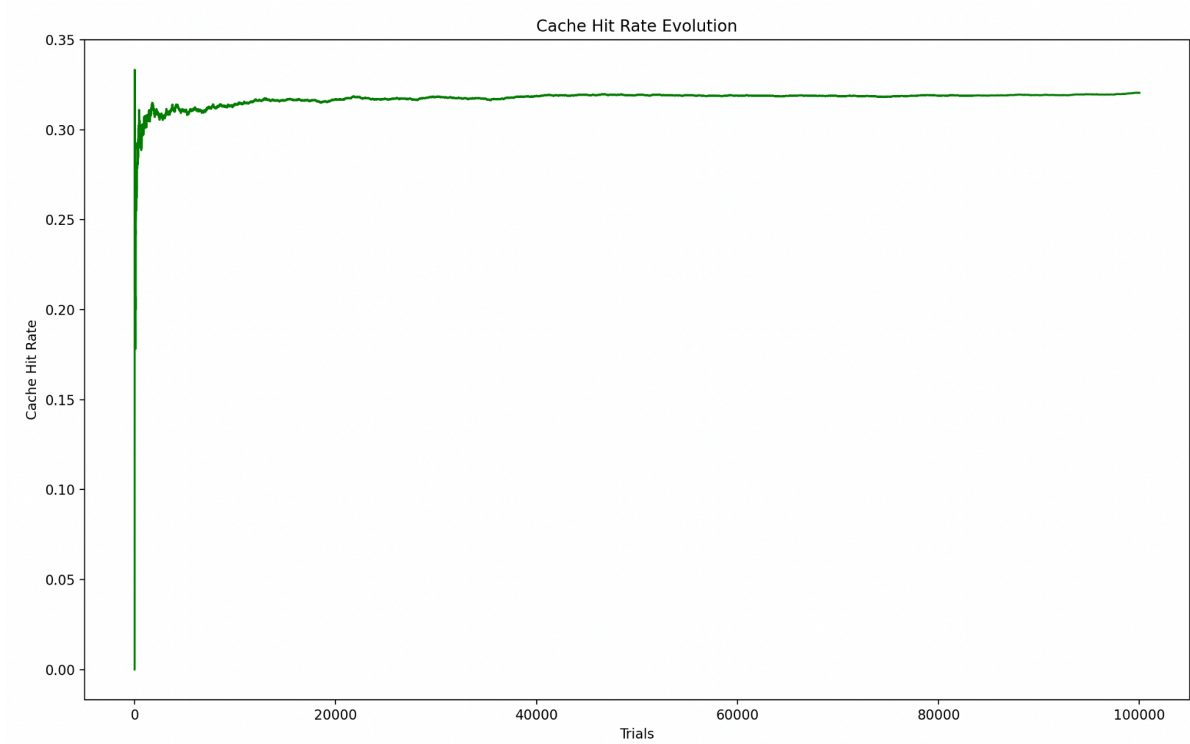
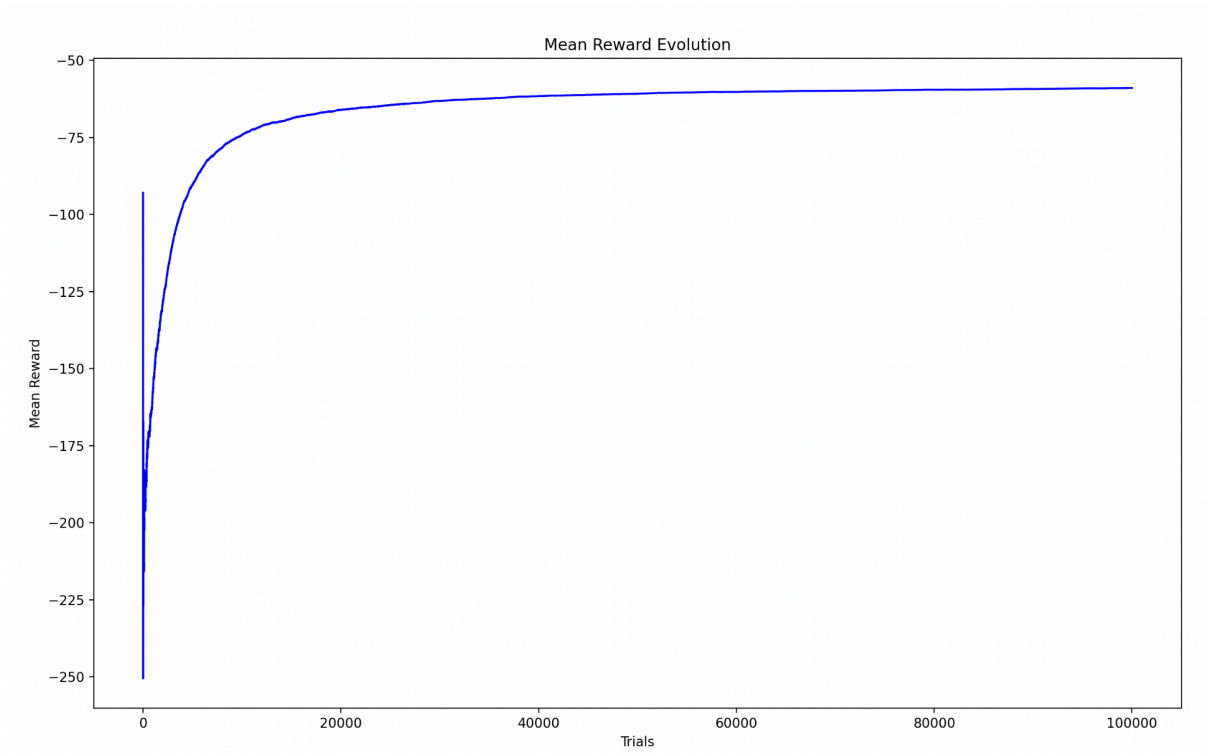
El programa imprimirá la información del sistema y del agente, luego entrenará el modelo. Al final, mostrará gráficas con la evolución de:

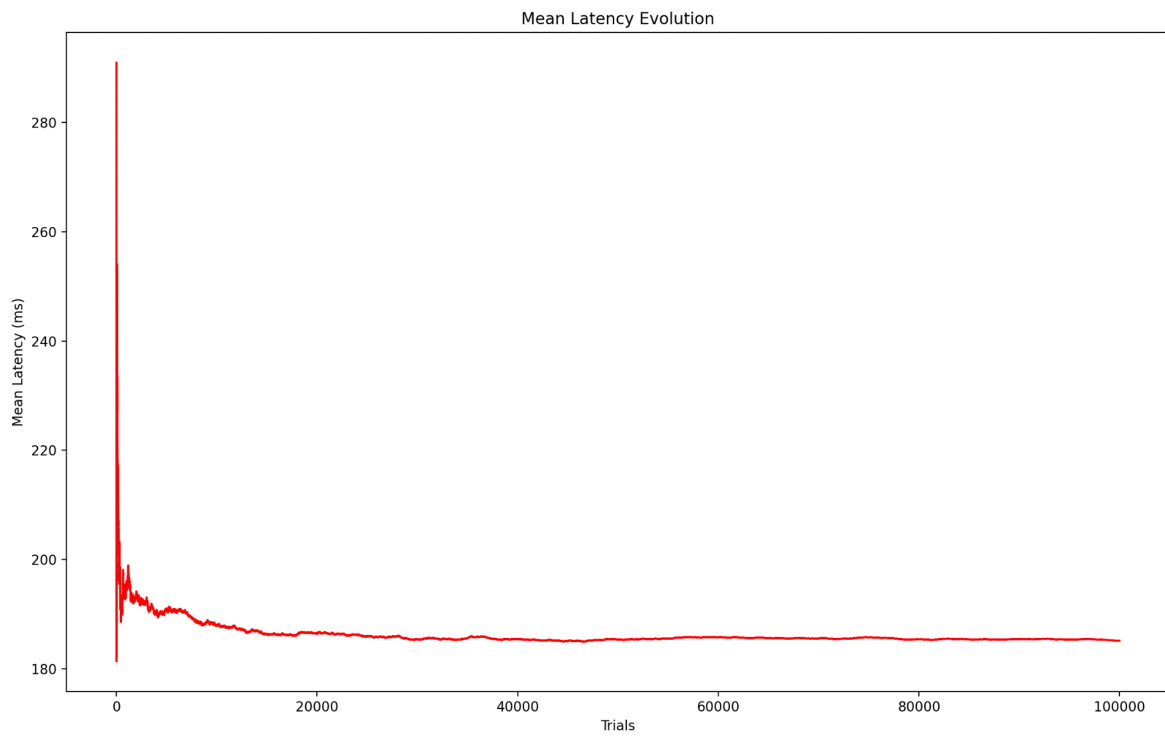
- **Recompensa media** → Cómo evoluciona la eficiencia de la caché con el tiempo.
- **Tasa de aciertos en caché** → La cantidad de veces que el objeto solicitado estaba en la caché.
- **Latencia media** → Tiempo medio para recuperar los objetos solicitados.

5. Resultados y Análisis

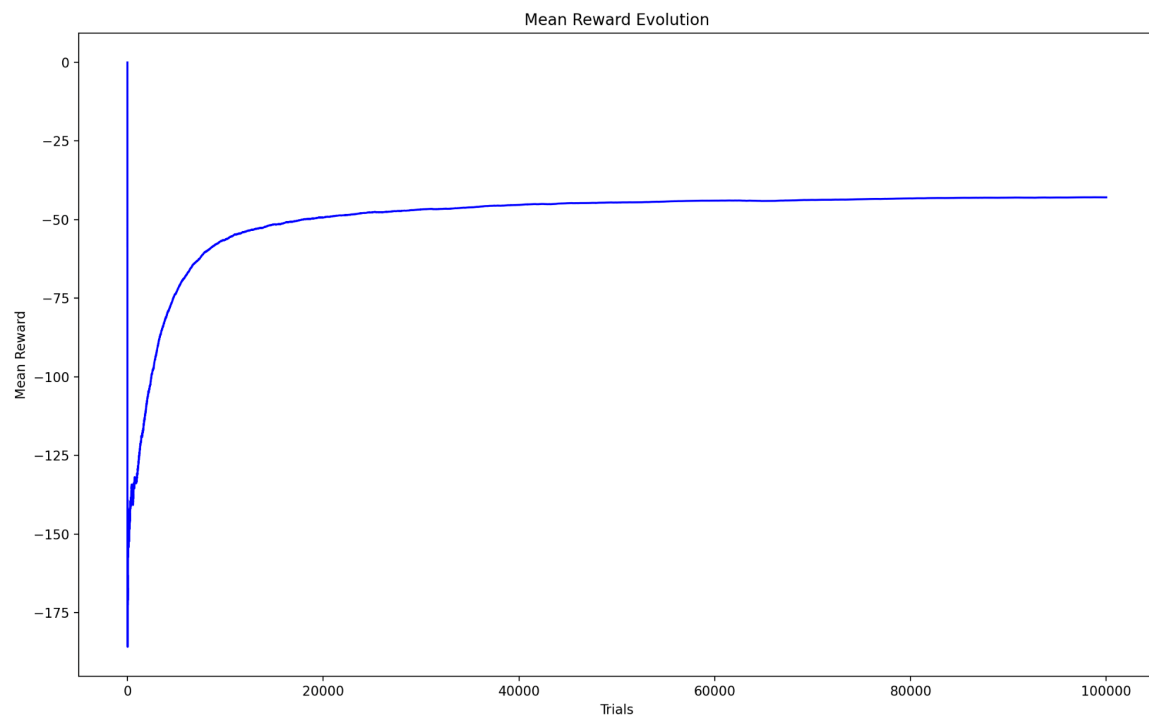
Una vez implementado el modelo se plantea una situación representativa que simule el tipo de tráfico al que el modelo tendría que enfrentarse en un entorno real. Para esto se toma un conjunto de objetos posibles de 50 objetos con latencias diferentes, ordenados de manera aleatoria (recordemos que el orden especifica las frecuencias, de mayor a menor). Se plantea que la memoria caché para esta situación tiene capacidad de 15 objetos. El factor de descuento utilizado es de 0.9, esto para que el agente prefiera encontrar políticas que obtengan buenas recompensas a largo plazo en lugar de una estrategia voraz. Con estos parámetros (el conjunto de objetos disponibles se encuentra en el código fuente de la aplicación), el agente es entrenado por 100000 iteraciones utilizando 3 semillas distintas: 1, 12 y 123 para la generación de números pseudo aleatorios. Los resultados para cada situación son los siguientes

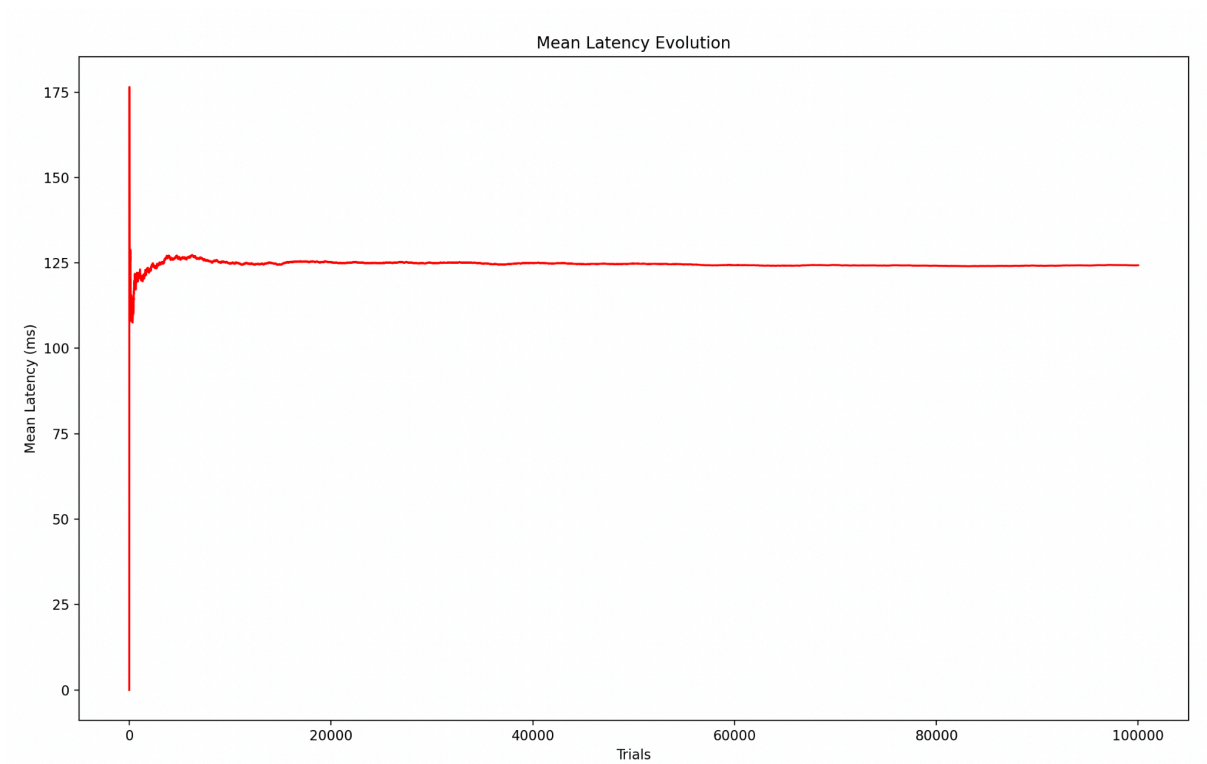
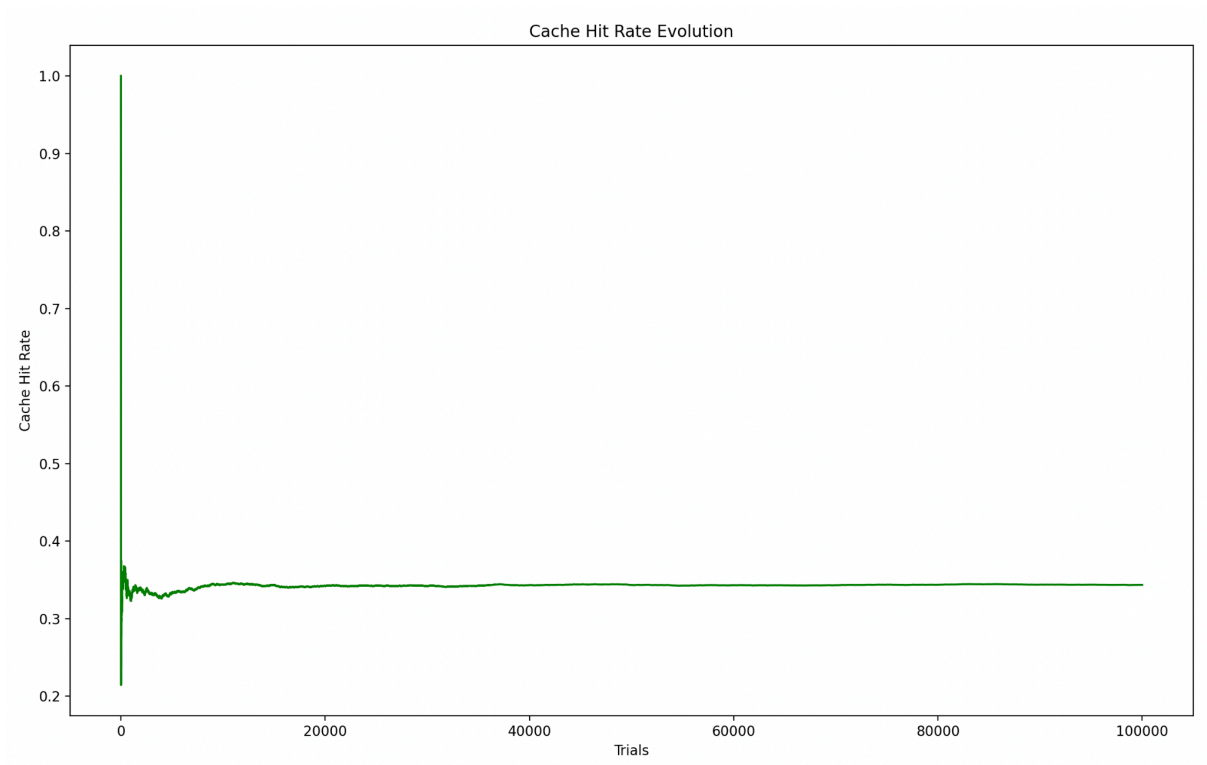
Semilla = 1



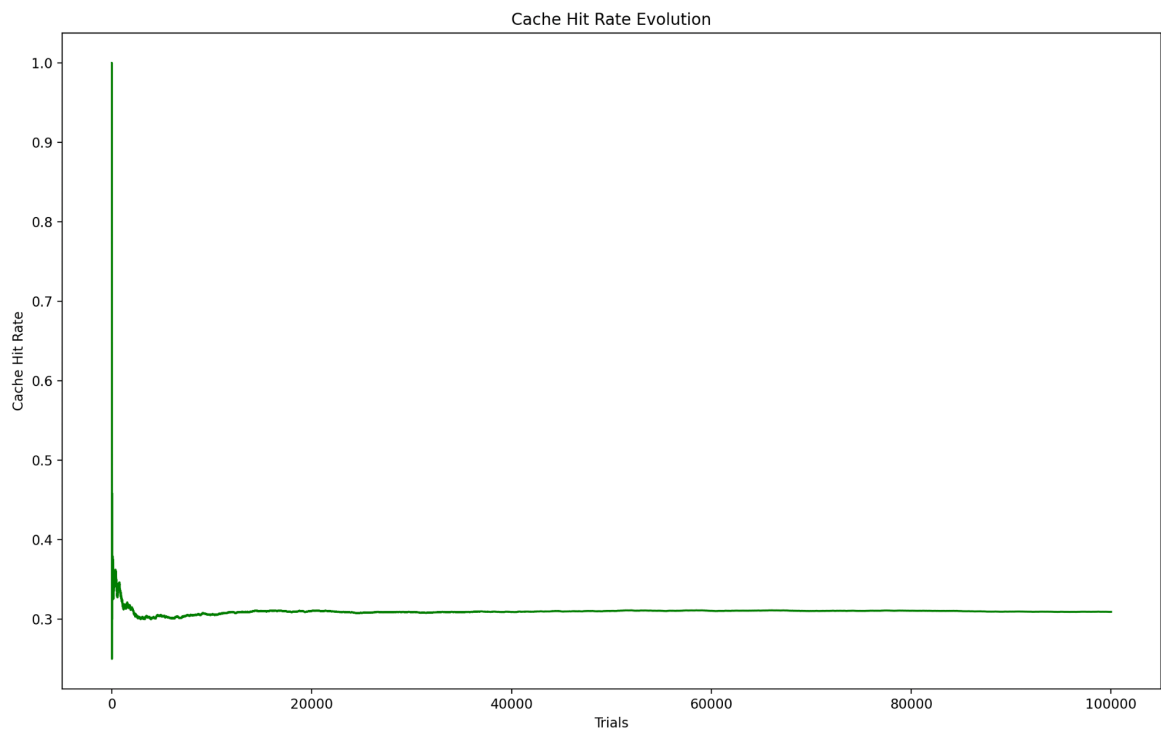
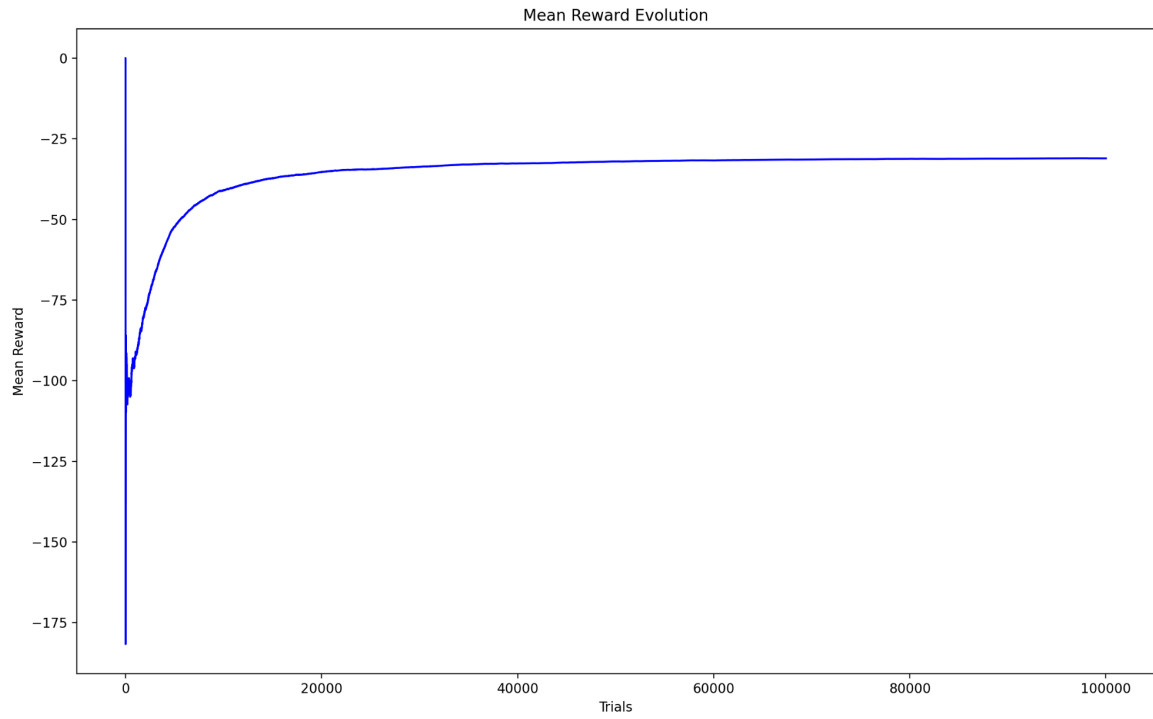


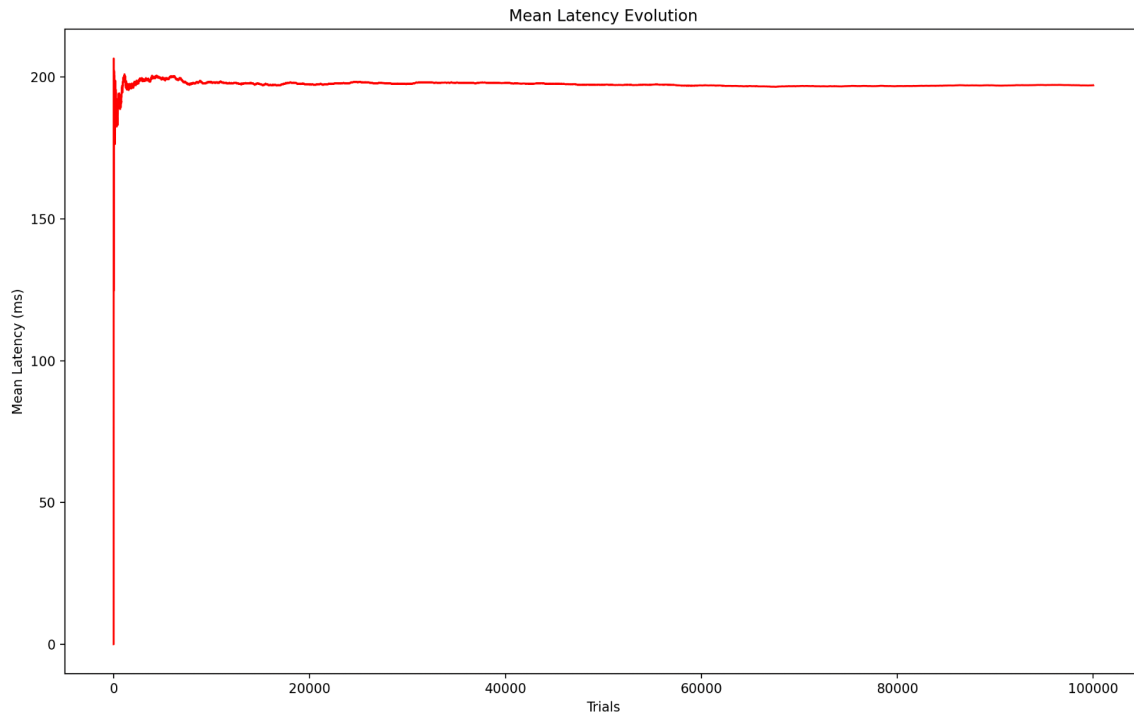
Semilla = 12





Semilla = 123





Las métricas se mantienen consistentes entre experimentos y se observa cómo a medida que avanzan las iteraciones el agente va refinando su política.

6. Conclusiones

- La baja tasa de hits se puede deber a la forma en que se entrena el agente, donde las recompensas son las latencias. El agente parece preferir almacenar objetos de gran latencia y alta frecuencia en la caché para reducir la latencia promedio. De esta forma, si necesita recuperar objetos con poca latencia, esto no tendrá una repercusión demasiado alta en su desempeño.
- Se puede observar cómo en las primeras iteraciones el desempeño es muy variable, esto es coherente con el hecho de que durante las primeras iteraciones se aplica mayormente la política de exploración aleatoria.
- La latencia promedio del sistema se ve reducida, por lo tanto, el agente termina aproximando una política lo suficientemente buena como para hacer frente a 50 objetos distintos con una caché de tan solo 15 objetos de capacidad. Si se incrementa esta capacidad, el agente podrá mejorar su desempeño aún más.

7. Referencias

- [1] S. P. Meyn and R. L. Tweedie, *Markov Chains and Stochastic Stability*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2009.
- [2] S. J. Russell y P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Upper Saddle River, NJ, USA: Pearson, 2020.
- [3] A. Bhattacharjee, M. Martonosi, and S. Malik, "Cache Replacement Policy Optimization via Reinforcement Learning," *IEEE Transactions on Computers*, vol. 64, no. 12, pp. 3820-3833, Dec. 2015.
- [4] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, "Web Caching and Zipf-like Distributions: Evidence and Implications," *IEEE INFOCOM*, vol. 1, pp. 126-134, 1999.
- [5] C. J. C. H. Watkins and P. Dayan, "Q-Learning," *Machine Learning*, vol. 8, no. 3-4, pp. 279-292, 1992.
- [6] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018.